



CrossTouch Input System — Version 1.1

Before using the CrossTouch system, ensure your Unity project is configured to use the **CrossTouch Input Asset** globally. This ensures consistent and reliable input mapping across all scenes and prefabs.

1. Tap Family

■ OnTap(Vector2 position)

Description: Fired when a single quick tap occurs. Commonly used for selecting or interacting with objects.

Parameters: Vector2 position — screen-space coordinate of the tap.

Returns: UnityEvent — invokes position of the tap.

When Triggered: When finger/mouse is pressed and released quickly within the tap threshold.

Example:

```
CrossTouchManager.Subscribe(OnTap, pos => { Debug.Log($"Tapped at {pos}"); });
```

■ OnDoubleTap(Vector2 position)

Description: Triggered when two taps occur within the double-tap interval.

Parameters: Vector2 position — position of the second tap.

Returns: UnityEvent

When Triggered: After detecting two taps within defined interval.

Example:

```
CrossTouchManager.Subscribe(OnDoubleTap, pos => { player.Jump(); });
```

■ OnTripleTap(Vector2 position)

Description: Triggered when three taps are detected quickly. Useful for hidden or debug actions.

Parameters: Vector2 position — position of last tap.

Returns: UnityEvent

When Triggered: After detecting three consecutive taps within threshold.

Example:

```
CrossTouchManager.Subscribe(OnTripleTap, pos => { ToggleDebugMode(); });
```

2. Hold Family

■ OnHoldStart(Vector2 startPos)

Description: Invoked when user starts holding a touch or button.

Parameters: Vector2 startPos — position where hold started.

Returns: UnityEvent

When Triggered: At the moment press duration starts being measured.

Example:

```
CrossTouchManager.Subscribe(OnHoldStart, pos => { ShowHoldIndicator(pos); });
```

■ OnHoldProgress(float normalized)

Description: Continuously updates during hold, providing normalized progress (0–1).

Parameters: float normalized — represents hold duration progress.

Returns: UnityEvent

When Triggered: While hold is in progress, fired every interval.

Example:

```
CrossTouchManager.Subscribe(OnHoldProgress, value => { progressBar.fillAmount = value; });
```

■ OnHoldComplete(Vector2 position)

Description: Triggered once when hold reaches completion threshold.

Parameters: Vector2 position — position of completed hold.

Returns: UnityEvent

When Triggered: When elapsed time >= hold threshold.

Example:

```
CrossTouchManager.Subscribe(OnHoldComplete, pos => { ChargeSkill(); });
```

■ OnHoldEnd(Vector2 position)

Description: Called when finger or button is released, regardless of completion.

Parameters: Vector2 position — release position.

Returns: UnityEvent

When Triggered: When hold ends or is canceled.

Example:

```
CrossTouchManager.Subscribe(OnHoldEnd, pos => { HideHoldIndicator(); });
```

3. Swipe Family

■ OnSwipeStart(Vector2 position)

Description: Called when a swipe gesture begins.

Parameters: Vector2 position — starting position of swipe.

Returns: UnityEvent

When Triggered: When user movement surpasses the minimum distance.

Example:

```
CrossTouchManager.Subscribe(OnSwipeStart, pos => { Debug.Log("Swipe started"); });
```

■ OnSwipeProgress(Vector2 direction)

Description: Continuously provides directional delta of the swipe.

Parameters: Vector2 direction — normalized movement direction.

Returns: UnityEvent

When Triggered: While finger continues swiping.

Example:

```
CrossTouchManager.Subscribe(OnSwipeProgress, dir => { arrowIndicator.RotateTowards(dir); });
```

■ OnSwipeEnd(Vector2 position)

Description: Called when swipe motion ends.

Parameters: Vector2 position — end position.

Returns: UnityEvent

When Triggered: When swipe stops or finger lifted.

Example:

```
CrossTouchManager.Subscribe(OnSwipeEnd, pos => { HideSwipeTrail(); });
```

■ OnSwipe(Vector2 finalDirection)

Description: Triggered after a valid swipe is completed. Returns final direction vector.

Parameters: Vector2 finalDirection — normalized vector of swipe.

Returns: UnityEvent

When Triggered: At swipe completion.

Example:

```
CrossTouchManager.Subscribe(OnSwipe, dir => { player.Move(dir); });
```

■ OnSwipeDirection(SwipeDirection dir)

Description: Provides 8-way directional classification of the swipe.

Parameters: SwipeDirection dir — Enum (Up, Down, Left, Right, etc.)

Returns: UnityEvent

When Triggered: After swipe direction is interpreted.

Example:

```
CrossTouchManager.Subscribe(OnSwipeDirection, dir => {  
    player.RotateTowards(dir); });
```

4. Drag Family

■ OnDragStart(Vector2, Ray)

Description: Invoked when dragging begins. Provides screen position and ray from camera.

Parameters: Vector2 screenPos, Ray ray

Returns: UnityEvent

When Triggered: Once movement exceeds drag threshold.

Example:

```
CrossTouchManager.Subscribe(OnDragStart, (pos, ray) => { Debug.Log($"Drag started at {pos}"); });
```

■ OnDragProgress(Vector2 delta)

Description: Continuous drag delta in pixels since last frame.

Parameters: Vector2 delta — change in position.

Returns: UnityEvent

When Triggered: While dragging continues.

Example:

```
CrossTouchManager.Subscribe(OnDragProgress, delta => { MoveObject(delta); });
```

■ OnDragDirection(Vector2 dir)

Description: Normalized direction of drag movement.

Parameters: Vector2 dir

Returns: UnityEvent

When Triggered: Every drag frame update.

Example:

```
CrossTouchManager.Subscribe(OnDragDirection, dir => { player.Face(dir); });
```

■ OnDragRay(Vector2, Ray)

Description: Fired each frame with a raycast for gameplay interactions.

Parameters: Vector2 pos, Ray ray

Returns: UnityEvent

When Triggered: During dragging when valid collider hit.

Example:

```
CrossTouchManager.Subscribe(OnDragRay, (pos, ray) => { if (Physics.Raycast(ray, out var hit)) hit.transform.Highlight(); });
```

■ OnDragEnd(Vector2, Ray)

Description: Called once when drag completes.

Parameters: Vector2 pos, Ray ray

Returns: UnityEvent

When Triggered: On release after dragging.

Example:

```
CrossTouchManager.Subscribe(OnDragEnd, (pos, ray) => { Debug.Log("Drag ended"); });
```

5. Zoom & Twist Family

■ OnPinchStart(float distance)

Description: Triggered when two fingers begin a pinch gesture.

Parameters: float distance — initial distance between fingers.

Returns: UnityEvent

When Triggered: When two fingers detected on screen.

Example:

```
CrossTouchManager.Subscribe(OnPinchStart, d => { Debug.Log($"Pinch start distance {d}"); });
```

■ OnPinchProgress(float scale)

Description: Provides current pinch zoom scale. Scale < 1 means zoom in, > 1 zoom out.

Parameters: float scale — zoom factor.

Returns: UnityEvent

When Triggered: As pinch gesture changes.

Example:

```
CrossTouchManager.Subscribe(OnPinchProgress, scale => { camera.Zoom(scale); });
```

■ OnPinchSpeed(float speed)

Description: Provides rate of change in pinch distance per frame.

Parameters: float speed

Returns: UnityEvent

When Triggered: While pinch is active.

Example:

```
CrossTouchManager.Subscribe(OnPinchSpeed, spd => { Debug.Log($"Zoom speed {spd}"); });
```

■ OnTwistProgress(float angle)

Description: Triggered when two fingers rotate relative to each other.

Parameters: float angle — delta in degrees (positive=clockwise).

Returns: UnityEvent

When Triggered: When twist detected.

Example:

```
CrossTouchManager.Subscribe(OnTwistProgress, angle => { RotateMap(angle); });
```

6. Movement Vector Family

■ OnLookDelta(Vector2 delta)

Description: Captures look direction deltas for camera or aim control.

Parameters: Vector2 delta — mouse/touch delta.

Returns: UnityEvent

When Triggered: Whenever player moves look control.

Example:

```
CrossTouchManager.Subscribe(OnLookDelta, delta => { camera.Rotate(delta); });
```

■ OnMoveVector(Vector2 vector)

Description: Captures directional movement from touch, keyboard, or stick.

Parameters: Vector2 vector — movement axes (-1 to 1).

Returns: UnityEvent

When Triggered: Whenever player moves movement input.

Example:

```
CrossTouchManager.Subscribe(OnMoveVector, vec => { character.Move(vec); });
```

7. Player (Gamepad/XBOX) Family

■ OnXBOX_A/B/X/Y_Pressed()

Description: Triggered when respective button is pressed.

Parameters: None

Returns: UnityEvent

When Triggered: Immediately on press.

Example:

```
CrossTouchManager.Subscribe(OnXBOX_A_Pressed, () => Fire());
```

■ OnXBOX_A/B/X/Y_Hold()

Description: Invoked every frame while the button is held down.

Parameters: None

Returns: UnityEvent

When Triggered: While button is pressed.

Example:

```
CrossTouchManager.Subscribe(OnXBOX_B_Hold, () => ChargeAttack());
```

■ OnXBOX_A/B/X/Y_Released()

Description: Triggered when button is released.

Parameters: None

Returns: UnityEvent

When Triggered: When button is lifted.

Example:

```
CrossTouchManager.Subscribe(OnXBOX_X_Released, () => Reload());
```

■ OnXBOX_LT/RT_Hold(float value)

Description: Continuously sends trigger pressure value (0–1).

Parameters: float value

Returns: UnityEvent

When Triggered: While trigger partially or fully pressed.

Example:

```
CrossTouchManager.Subscribe(OnXBOX_RT_Hold, val => { enginePower = val; });
```

■ OnXBOX_LT/RT_Pressed()

Description: Fires once when trigger crosses threshold.

Parameters: None

Returns: UnityEvent

When Triggered: At initial trigger press.

Example:

```
CrossTouchManager.Subscribe(OnXBOX_LT_Pressed, () => AimDown());
```

■ OnXBOX_LT/RT_Released()

Description: Fires when trigger released below threshold.

Parameters: None

Returns: UnityEvent

When Triggered: When trigger released.

Example:

```
CrossTouchManager.Subscribe(OnXBOX_RT_Released, () => StopShoot());
```

CrossTouch Input System Summary

This deluxe documentation provides detailed descriptions, return values, and usage scenarios for every event available in the CrossTouch Input System. It covers tap, hold, swipe, drag, zoom, movement, and full gamepad support — ensuring developers can integrate the system seamlessly into any gameplay or editor environment. For future updates and technical support, join our official Discord community.

Join our official Discord community for support and updates:

<https://discord.com/invite/BTdKPfyr>